

LibroSystemRR

Semana 2 — Planificación y Diseño

Estructura de Datos y Algoritmos I

Escuela Politécnica Nacional — FIS

Integrantes:	Sebastián Ríos, Ariel Recalde
Periodo:	Mayo – Julio 2026
Entrega:	Viernes 5 de junio de 2026

Tabla de Contenidos

- 1. [Requerimientos Funcionales](#)
- 2. [Cronograma](#)
- 3. [Diagrama UML](#)
- 4. [Arquitectura a Utilizar](#)
- 5. [Buenas Prácticas de Programación](#)
- 6. [Prototipo](#)

1. Requerimientos Funcionales

Los requerimientos funcionales describen las funcionalidades que el sistema **debe cumplir** para resolver la problemática planteada.

1.1 RF-01 — Módulo de Catálogo

ID	Requerimiento	Prioridad
RF-01-01	El sistema debe permitir registrar un libro con: ISBN, título, autor, año de publicación y disponibilidad.	Alta
RF-01-02	El sistema debe permitir buscar un libro por ISBN utilizando búsqueda en BST.	Alta
RF-01-03	El sistema debe permitir buscar libros por título utilizando búsqueda lineal sobre lista enlazada.	Alta
RF-01-04	El sistema debe permitir eliminar un libro del catálogo por ISBN.	Media
RF-01-05	El sistema debe permitir ordenar el catálogo por título usando MergeSort.	Alta
RF-01-06	El sistema debe permitir ordenar el catálogo por autor usando MergeSort.	Media

ID	Requerimiento	Prioridad
RF-01-07	El sistema debe listar todos los libros disponibles en orden inorden del BST.	Alta

1.2 RF-02 — Módulo de Usuarios

ID	Requerimiento	Prioridad
RF-02-01	El sistema debe permitir registrar usuarios de tipo Lector, Bibliotecario o Ayudante.	Alta
RF-02-02	El sistema debe permitir buscar un usuario por ID mediante búsqueda lineal.	Alta
RF-02-03	El sistema debe validar que un Lector no supere el límite de 3 préstamos activos.	Alta
RF-02-04	El sistema debe validar que un Lector sin deudas pueda realizar un nuevo préstamo.	Alta
RF-02-05	El sistema debe mostrar el historial de préstamos de un usuario usando la Pila.	Media

1.3 RF-03 — Módulo de Préstamos

ID	Requerimiento	Prioridad
RF-03-01	El sistema debe permitir registrar un préstamo asociando un usuario y un libro disponible.	Alta
RF-03-02	El sistema debe actualizar la disponibilidad del libro al registrar un préstamo.	Alta
RF-03-03	El sistema debe gestionar la cola de préstamos activos de cada usuario (FIFO).	Alta
RF-03-04	El sistema debe permitir procesar la devolución de un préstamo.	Alta
RF-03-05	El sistema debe calcular los días de retraso al procesar una devolución.	Alta
RF-03-06	El sistema debe registrar cada operación en el historial usando la Pila.	Media

1.4 RF-04 — Módulo de Alertas y Multas

ID	Requerimiento	Prioridad
RF-04-01	El sistema debe detectar préstamos vencidos comparando la fecha actual con la fecha de devolución.	Alta

ID	Requerimiento	Prioridad
RF-04-02	El sistema debe generar una multa automáticamente cuando se detecte un préstamo vencido.	Alta
RF-04-03	El sistema debe calcular el monto de la multa a razón de \$0.50 por día de retraso.	Alta
RF-04-04	El sistema debe permitir registrar el pago de una multa.	Alta
RF-04-05	El sistema debe listar todos los préstamos vencidos ordenados por fecha de vencimiento.	Media

2. Cronograma

El proyecto se desarrolla en 7 semanas con cierre cada viernes. Las semanas 3, 4 y 5 corresponden a iteraciones de desarrollo incremental.

2.1 Resumen de Fases y Fechas

S	Fase	Inicio	Cierre	Entregables
S1	Inicio del Proyecto	26/05/2026	29/05/2026	Acta de constitución · Stakeholders
S2	Planificación y Diseño	01/06/2026	05/06/2026	Requerimientos · Cronograma · UML · Arquitectura · Buenas prácticas · Prototipo
S3	Iteración 1 — TADs	08/06/2026	12/06/2026	Avance funcional 1: TADs + Modelos base
S4	Iteración 2 — Algoritmos	15/06/2026	19/06/2026	Avance funcional 2: Algoritmos + Préstamos
S5	Iteración 3 — UI	22/06/2026	26/06/2026	Avance funcional 3: Sistema completo con UI
S6	Integración y Calidad	29/06/2026	03/07/2026	Sistema integrado · Informe técnico
S7	Cierre y Defensa	06/07/2026	10/07/2026	Sistema completo · Doc. técnico · Defensa

2.2 Matriz RACI — Leyenda

La Matriz RACI define el nivel de participación de cada integrante en cada actividad del proyecto.

Código	Significado
R	Responsable — ejecuta la tarea directamente
A	Aprobador — revisa y da el visto bueno
C	Consultado — aporta criterio o apoyo técnico
I	Informado — recibe actualizaciones del avance

2.3 Actividades Detalladas por Semana con RACI

Cada actividad tiene asignada una fecha concreta dentro de la semana correspondiente, distribuida de lunes a jueves para reservar el viernes como día de entrega y revisión.

Semana 1 — Inicio del Proyecto | 26/05/2026 → 29/05/2026

Actividad	Fecha	Entregable	Sebastián	Ariel
Definir problemática del sistema	26/05/2026	Acta de constitución	R	A
Seleccionar temática del proyecto	26/05/2026	Acta de constitución	A	R
Redactar objetivos SMART	27/05/2026	Acta de constitución	R	C
Definir alcance inicial	27/05/2026	Acta de constitución	C	R
Identificar y documentar stakeholders	28/05/2026	Matriz de stakeholders	R	R
Asignar roles del equipo	28/05/2026	Acta de constitución	A	R

Semana 2 — Planificación y Diseño | 01/06/2026 → 05/06/2026

Actividad	Fecha	Entregable	Sebastián	Ariel
Definir RF-01 a RF-04	01/06/2026	Tabla de requerimientos	R	A
Diseñar diagrama UML de clases	01/06/2026	uml_librosystemrr.puml	C	R
Definir arquitectura N-Capas	02/06/2026	Sección arquitectura	R	C
Documentar buenas prácticas y SOLID	02/06/2026	Sección buenas prácticas	A	R
Elaborar cronograma con RACI	03/06/2026	Cronograma + RACI	R	C
Diseñar prototipo UI estático	04/06/2026	Prototipo.html	R	R

Semana 3 — Iteración 1: TADs y Modelos | 08/06/2026 → 12/06/2026

Actividad	Fecha	Entregable	Sebastián	Ariel
Implementar <code>Nodo<T></code> y <code>ListaEnlazada<T></code>	08/06/2026	Avance funcional 1	R	I
Implementar <code>Cola<T></code> y <code>Pila<T></code>	09/06/2026	Avance funcional 1	R	I
Implementar <code>NodoBST</code> y <code>CatalogoBST</code>	09/06/2026	Avance funcional 1	C	R
Implementar modelo <code>Libro</code> con <code>Ordenable<T></code>	10/06/2026	Avance funcional 1	R	C
Implementar <code>Usuario</code> y subclase <code>Lector</code>	10/06/2026	Avance funcional 1	R	C
Pruebas unitarias de TADs	11/06/2026	Avance funcional 1	R	R

Semana 4 — Iteración 2: Algoritmos y Préstamos | 15/06/2026 → 19/06/2026

Actividad	Fecha	Entregable	Sebastián	Ariel
Implementar AlgoritmosBusqueda	15/06/2026	Avance funcional 2	R	C
Implementar AlgoritmosOrdenamiento	16/06/2026	Avance funcional 2	C	R
Implementar SistemaBiblioteca	16/06/2026	Avance funcional 2	C	R
Implementar módulo de préstamos (Prestamo.java)	17/06/2026	Avance funcional 2	C	R
Implementar Bibliotecario y AyudanteBibliotecario	17/06/2026	Avance funcional 2	R	C
Pruebas integración algoritmos + sistema	18/06/2026	Avance funcional 2	R	R

Semana 5 — Iteración 3: UI y Multas | 22/06/2026 → 26/06/2026

Actividad	Fecha	Entregable	Sebastián	Ariel
Implementar módulo de alertas y multas	22/06/2026	Avance funcional 3	C	R
Implementar VentanaPrincipal con Swing	22/06/2026	Avance funcional 3	R	R
Implementar PanelCatalogo y PanelPrestamos	23/06/2026	Avance funcional 3	R	C
Implementar PanelUsuarios y PanelAlertas	24/06/2026	Avance funcional 3	C	R
Conectar UI con SistemaBiblioteca	24/06/2026	Avance funcional 3	R	R
Implementar diálogos (Libro, Usuario, Préstamo)	25/06/2026	Avance funcional 3	R	R

Semana 6 — Integración y Calidad | 29/06/2026 → 03/07/2026

Actividad	Fecha	Entregable	Sebastián	Ariel
Integrar todos los módulos	29/06/2026	Sistema integrado	R	R
Ejecutar pruebas funcionales end-to-end	30/06/2026	Sistema integrado	R	A

Actividad	Fecha	Entregable	Sebastián	Ariel
Cargar datos de prueba (<code>cargarDatosDePrueba()</code>)	30/06/2026	Sistema integrado	C	R
Analizar complejidad de algoritmos	01/07/2026	Informe técnico	R	R
Redactar tabla de complejidades Mejor/Prom/Peor	01/07/2026	Informe técnico	A	R
Validar <code>SistemaLogger</code> y excepciones	02/07/2026	Sistema integrado	R	C

Semana 7 — Cierre y Defensa | 06/07/2026 → 10/07/2026

Actividad	Fecha	Entregable	Sebastián	Ariel
Preparar presentación final	06/07/2026	Presentación	R	R
Redactar documento técnico final	07/07/2026	Documento técnico	R	R
Preparar defensa oral (Sebastián)	07/07/2026	Defensa oral	R	I
Preparar defensa oral (Ariel)	08/07/2026	Defensa oral	I	R
Revisión final y cierre del proyecto	09/07/2026	Sistema completo	R	R

2.4 Distribución de Módulos por Integrante

Módulo / Componente	Responsable Principal	Apoyo
TADs lineales (Nodo, Lista, Cola, Pila)	Sebastián Ríos	Ariel Recalde
TAD árbol (NodoBST, CatalogoBST)	Ariel Recalde	Sebastián Ríos
AlgoritmosBusqueda	Sebastián Ríos	Ariel Recalde
AlgoritmosOrdenamiento	Ariel Recalde	Sebastián Ríos
Modelos (Libro, Usuario, subclases)	Sebastián Ríos	Ariel Recalde
Prestamo, Multa, SistemaBiblioteca	Ariel Recalde	Sebastián Ríos
UI Swing (paneles y diálogos)	Ambos	—
Documentación e informe técnico	Ambos	—

3. Diagrama UML

El diagrama de clases fue elaborado en PlantUML (archivo: `uml_librosystemrr.puml`) e incluye:

- 11 paquetes organizados por capas.
- TADs con `Nodo<T>` explícito como base interna de `ListaEnlazada<T>`, `Cola<T>` y `Pila<T>`.
- Clases `AlgoritmosBusqueda` y `AlgoritmosOrdenamiento` con complejidades anotadas.
- Métodos recursivos marcados con estereotipo «`recursive`».

- Relaciones de herencia, composición, dependencia e implementación.
- Patrón Singleton en **SistemaLogger** con nota explicativa.

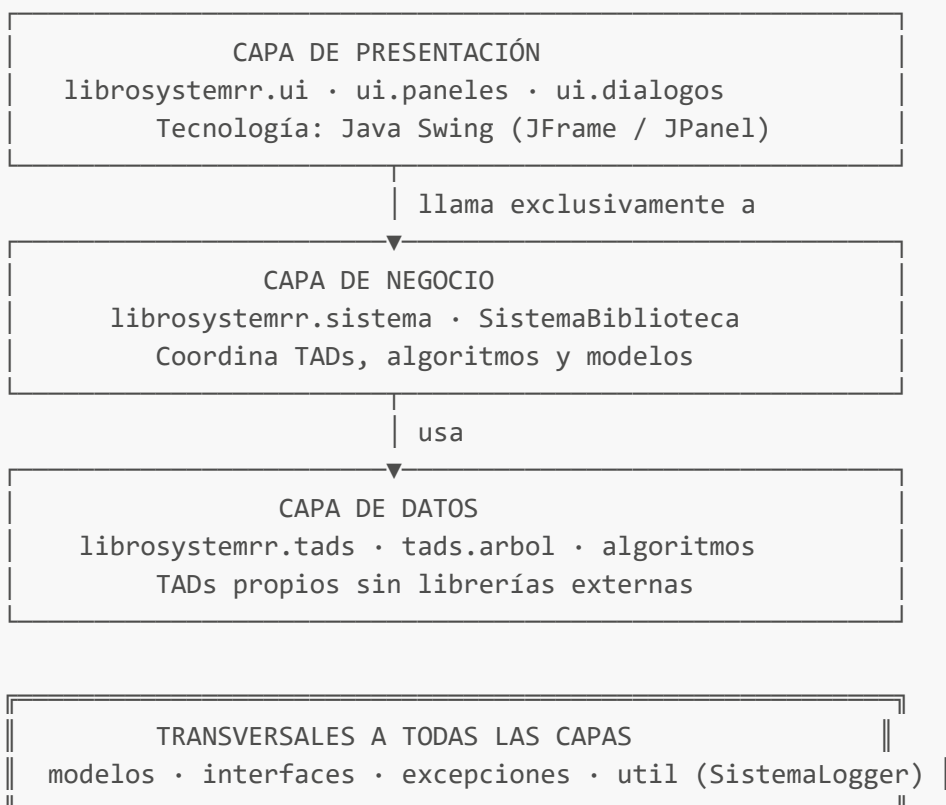
Estructura de paquetes del diagrama:

<code>librosystemrr.interfaces</code>	-> <code>Buscable</code> , <code>Ordenable<T></code>
<code>librosystemrr.excepciones</code>	-> <code>LibroSystemException</code> y subclases
<code>librosystemrr.util</code>	-> <code>SistemaLogger</code> (Singleton)
<code>librosystemrr.tads</code>	-> <code>Nodo<T></code> , <code>ListaEnlazada<T></code> , <code>Cola<T></code> , <code>Pila<T></code>
<code>librosystemrr.tads.arbol</code>	-> <code>NodoBST</code> , <code>CatalogoBST</code>
<code>librosystemrr.algoritmos</code>	-> <code>AlgoritmosBusqueda</code> , <code>AlgoritmosOrdenamiento</code>
<code>librosystemrr.modelos</code>	-> <code>Libro</code> , <code>Usuario</code> , <code>Lector</code> , <code>Bibliotecario</code> , <code>AyudanteBibliotecario</code> , <code>Prestamo</code> , <code>Multa</code>
<code>librosystemrr.sistema</code>	-> <code>SistemaBiblioteca</code>
<code>librosystemrr.ui</code>	-> <code>VentanaPrincipal</code>
<code>librosystemrr.ui.paneles</code>	-> <code>PanelCatalogo</code> , <code>PanelPrestamos</code> , <code>PanelUsuarios</code> , <code>PanelAlertas</code>
<code>librosystemrr.ui.dialogos</code>	-> <code>DialogoLibro</code> , <code>DialogoUsuario</code> , <code>DialogoPrestamo</code>

4. Arquitectura a Utilizar

4.1 Patrón: Arquitectura N-Capas

Se adoptó la arquitectura de **N capas** para separar responsabilidades y garantizar que cada módulo tenga una única función dentro del sistema.



4.2 Comparativa de Arquitecturas

Alternativa	Justificación de descarte / selección
Monolítica sin capas	Mezcla lógica de negocio con UI y estructuras. Difícil de mantener y probar.
MVC	Más adecuado para aplicaciones web. Para Swing, N-Capas es más natural y simple.
Microservicios	Excesivo para un sistema de escritorio académico sin red.
N-Capas ✓ (seleccionada)	Separa presentación, negocio y datos. Cada capa tiene una única responsabilidad. Facilita la defensa técnica.

4.3 Reglas de Comunicación entre Capas

- La capa de **presentación** solo llama a `SistemaBiblioteca`. Nunca accede directamente a TADs.
- La capa de **negocio** coordina TADs y algoritmos. No conoce la existencia de la UI.
- La capa de **datos** (TADs) es completamente genérica `<T>`. No conoce a `Libro` ni a `Usuario`.

5. Buenas Prácticas de Programación

5.1 Convenciones de Nomenclatura (Java Standard)

Elemento	Convención	Ejemplo
Clases	PascalCase	<code>ListaEnlazada</code> , <code>CatalogoBST</code>
Interfaces	PascalCase	<code>Buscable</code> , <code>Ordenable</code>
Métodos	camelCase	<code>buscarPorIsbn()</code> , <code>mergeSort()</code>
Variables	camelCase	<code>cabeza</code> , <code>tamano</code> , <code>fechaDevolucion</code>
Constantes	UPPER_SNAKE_CASE	<code>MONTO_POR_DIA</code> , <code>LIMITE_DEFAULT</code>
Paquetes	minúsculas sin guiones	<code>librosystemrr.tads.arbol</code>
Archivos	igual que la clase pública	<code>CatalogoBST.java</code>

5.2 Principios SOLID Aplicados

S — Responsabilidad Única (SRP)

- `AlgoritmosBusqueda` solo busca, `AlgoritmosOrdenamiento` solo ordena, `SistemaLogger` solo registra eventos.

O — Principio Abierto/Cerrado (OCP)

- La clase `Usuario` está abierta para extensión (subclases `Lector`, `Bibliotecario`, `AyudanteBibliotecario`) y cerrada para modificación.

L — Sustitución de Liskov (LSP)

- Cualquier subclase de `Usuario` puede usarse donde se espere un `Usuario` sin romper el sistema.

I — Segregación de Interfaces (ISP)

- Interfaces específicas: `Buscable` para búsqueda y `Ordenable<T>` para comparación. Ninguna clase implementa métodos que no necesita.

D — Inversión de Dependencias (DIP)

- `SistemaBiblioteca` depende de abstracciones (`Buscable`, `Ordenable`) y no de implementaciones concretas.

5.3 Patrones de Diseño Aplicados

Patrón	Aplicación	Justificación
Singleton	<code>SistemaLogger</code>	Una sola instancia del logger en todo el sistema.
Template Method	<code>Usuario</code> (abstract)	Define el esqueleto del comportamiento. Subclases implementan <code>getTipo()</code> .
Factory Method	<code>DialogoUsuario</code>	Crea instancias de <code>Lector</code> , <code>Bibliotecario</code> o <code>Ayudante</code> según selección.

5.4 Otras Prácticas Aplicadas

- **Una clase pública por archivo** — estándar de Java.
- **Encapsulamiento estricto** — todos los atributos son `private` o `protected`. Acceso solo mediante getters/setters.
- **Genéricos <T>** — los TADs son completamente genéricos, reutilizables con cualquier tipo.
- **Excepciones personalizadas** — se lanza `LibroNoEncontradoException` en lugar de retornar `null`.
- **Javadoc** — todos los métodos públicos documentados con `@param`, `@return` y `@throws`.
- **Datos de prueba** — `cargarDatosDePrueba()` en `SistemaBiblioteca` para facilitar la defensa.
- **Sin librerías externas para TADs** — `ArrayList`, `LinkedList`, `Stack` de Java no se usan. Todo implementado desde cero.

5.5 Análisis de Complejidad

La tabla completa de complejidades Mejor/Promedio/Peor caso se encuentra en el Informe Técnico ([docs/informe_tecnico.pdf](#)). Resumen de decisiones:

Estructura / Algoritmo	Complejidad	Justificación
<code>CatalogoBST</code> — buscar	$O(\log n)$ prom.	Más eficiente que lista $O(n)$ para catálogos grandes.
MergeSort	$O(n \log n)$	Estable, ideal para <code>ListaEnlazada</code> . Sin acceso aleatorio.
QuickSort	$O(n \log n)$ prom.	In-place, ideal para arreglos <code>Libro[]</code> .

Estructura / Algoritmo	Complejidad	Justificación
<code>Cola<Prestamo></code> — encolar	$O(1)$	FIFO natural para gestión de préstamos.
<code>Pila<String></code> — apilar	$O(1)$	LIFO natural para historial de operaciones.

6. Prototipo

El prototipo es una maqueta visual estática de la interfaz de usuario del sistema **LibroSystemRR**. Muestra la estructura de navegación y los paneles principales que serán implementados en Java Swing.

Archivo adjunto: `LibroSystemRR_Prototipo.html`

Pantallas incluidas en el prototipo:

- **Panel de Catálogo** — búsqueda y listado de libros.
- **Panel de Préstamos** — registro y devolución.
- **Panel de Usuarios** — gestión de lectores.
- **Panel de Alertas** — libros vencidos y multas.